Coding Guidelines for Participants

TalentAP Mega Hackathon 2026 · Retail · Healthcare · BFSI

v1.1 · for all participating teams

Your code matters, not just your demo. The code you submit is reviewed for how well it is written, structured and secured, as well as for what it does. Follow these guidelines while you build. They are simply good engineering habits that apply to any language or framework.

Use any stack
Python, JavaScript/TypeScript, Java, Go, Rust, .NET or a mix. Pick what your team knows best.

AI is welcome
LLM APIs, local models, agents, MCP, cloud services and AI coding assistants are all allowed, never required.

Pre-built code: this hackathon only
You may write code for this hackathon before Day 1; declare it (rule 12). Libraries and templates are fine. Code from other hackathons or past projects is **not**.

How to submit
Upload one `.zip` named `teamID_psID.zip`, with all your code in `FULL_CODE/`. Use the checklist on page 4. The organisers announce when.

1 Write code others can read

- Use clear names: `calculate_premium()`, not `calc2()` or `doStuff()`.
- Keep functions short, with one job each. Avoid deep nesting and very long files.
- Run your language's formatter and linter (table on page 4) and **fix** the warnings instead of switching them off.
- Don't repeat the same logic in several places. Move it into a function.
- Use type hints (Python) or TypeScript types where your language supports them.

✗ Leave no dead code, commented-out blocks, unused files or debug prints.

2 Give your project a clear structure

- **Put all your code in one folder named `FULL_CODE/`**: your demo app plus the code that creates your submission files. Framework files and lockfiles go in too (e.g. `FULL_CODE/web/package.json`). Next to it, keep only `README.md`, `submission/` (Retail, Healthcare), `.gitignore` and `LICENSE`. **See the picture on page 2.**
- Keep the UI, API routes, business logic, data access and AI calls in separate modules or folders.
- Have one obvious entry point in `FULL_CODE/`, such as `main.py`, `src/index.ts`, `Main.java` or `Program.cs`.
- Put settings in one config file or in environment variables: URLs, ports, model names, thresholds and feature flags.
- Keep your core logic separate from the web framework so you can test it on its own.
- Keep your database schema in the repo as migration or SQL files (for example Prisma, Alembic, Flyway, EF Core migrations or a plain `schema.sql`), plus a small seed script. Don't only create tables by hand in a database console.

✗ Don't commit database files or dumps (`.db`, `.sqlite`, SQL exports with data).

✗ No giant single file that does everything, and no hard-coded absolute paths.

3 Keep secrets out of your code

- Never hard-code API keys, passwords, tokens or database connection strings.
- Read secrets from environment variables. Commit a `.env.example` with dummy values, and add `.env` to `.gitignore`.
- **Deleting a key in a later commit does not remove it from Git history.** If a key was ever committed, revoke it and create a new one.

✗ Never put keys in frontend code or in `NEXT_PUBLIC_` / `VITE_` variables. Call LLMs and paid APIs from your backend.

✗ Never print secrets, tokens or full personal records in logs or error messages.

4 Write secure code

- Validate every input from users, files and external APIs: its type, length, range and allowed values.
- Use parameterised queries or an ORM. Never build SQL by joining strings.
- Check permissions on the **server**. Hiding a button in the UI is not access control.
- Restrict CORS, turn off debug mode, and keep SSL certificate checks switched on.
- Using Supabase or Firebase? Keep row-level security / security rules on, and keep service keys on the server.

✗ Never pass user input (or LLM output) to `eval`, `exec`, shell commands or unsafe deserialisation such as `pickle`.

RULE 2 IN A PICTURE: YOUR PROJECT AT A GLANCE

Your zip is one fresh `git clone` of your project. At the top there are only four things: **your** `README.md`, `.gitignore`, `submission/` (Retail and Healthcare) and `FULL_CODE/`, plus an optional `LICENSE`. **Everything you wrote goes inside `FULL_CODE/`.** The folders inside `FULL_CODE/` below are only an example (a Python model, an API and a web demo); organise yours in whatever way suits your stack.

```
T042_H3.zip                          ← one zip, named teamID_psID.zip (use your own team and problem ID)
└── T042_H3/                         ← your project: a fresh git clone (it also holds the hidden .git folder)
    ├── README.md                    ← YOUR own README, not the pack's (rule 12; skeleton below)
    ├── .gitignore                   ← must list .env, data/, models/, .venv/, node_modules/ (and build output)
    ├── LICENSE                      ← optional
    ├── submission/                  ← Retail and Healthcare only: the exact file names from your problem's README
    │   └── H3_predictions.csv       ← commit it; make sure .gitignore does not exclude it
    └── FULL_CODE/                   ← ALL your code: your demo app + the code that created the submission/ files
        ├── .env.example             ← dummy values only; your real .env is never committed (rule 3)
        ├── pipeline/                ← example: the code that makes your submission/ files
        │   ├── pyproject.toml       ← dependencies; commit the lockfile (uv.lock) too (rule 5)
        │   ├── config.yaml          ← settings: paths, seeds, thresholds, model names (rule 2)
        │   ├── src/readmit/         ← features.py, train.py, predict.py (one obvious entry point)
        │   └── tests/               ← tests for your core logic (rule 6)
        ├── api/                     ← example: business logic kept separate from the web framework (rule 2)
        ├── web/                     ← example: your demo app; its package.json + lockfile stay in its own folder
        ├── db/                      ← schema.sql or migrations + a small seed script, if you use a database (rule 2)
        ├── data/                    ← the pack's data goes here on your laptop: listed in .gitignore, NOT committed
        └── models/                  ← downloaded models: listed in .gitignore, NOT committed (say in README how to get them)
```

Other stacks: same idea

- Next.js or React only: `FULL_CODE/web/package.json`
- Spring Boot: `FULL_CODE/backend/pom.xml`
- Android or Flutter: `FULL_CODE/mobile/` (the whole app project)
- .NET: `FULL_CODE/Api/Api.csproj`
- Notebooks: `FULL_CODE/notebooks/`, with reusable code in `FULL_CODE/src/` (rule 11)
- One small script is fine too: `FULL_CODE/main.py` plus its lockfile

BFSI teams

- Same layout, **without** `submission/`: there are no prediction files.
- B1: keep your schema, migrations and masking code in `FULL_CODE/`. Don't commit the pack's `.db` file or SQL dumps; add them to `.gitignore`.
- B2: load the pack's CSV/JSON files from a `data/` folder that is listed in `.gitignore`.
- Your README says where to put the pack's files.

Never in your zip

- ✗ Code outside `FULL_CODE/`, or a second copy of your project
- ✗ The problem pack's README instead of your own
- ✗ `.env`, keys or passwords
- ✗ The pack's data files, downloaded models, `.db` files or dumps
- ✗ `.venv`, `node_modules`, build output, zips inside your zip

A README skeleton for the top of your project (rule 12)

`# T042 · H3: 30-Day Readmission Risk`	your team ID, problem ID and a one-line summary
`## What it does`	the problem, who uses your solution, what they see
`## How to run`	setup and run commands, environment variables (see `FULL_CODE/.env.example`), where to put the data
`## How to run the tests`	the one command that runs them
`## Architecture`	a short diagram or a list of the main parts inside `FULL_CODE/` and how they connect
`## AI usage`	models, APIs and agents in your app, plus the AI coding assistants you used
`## Pre-built and third-party code`	what you wrote before Day 1; templates, starter kits and copied snippets (with licences)
`## Limitations`	known issues and what you would do next. Be honest.

How to make the zip: commit everything, run `git clone <your project> T042_H3` in the folder above your project, open the new folder to check it looks like the picture above, then zip it. The Data Guide on the download page (Step 8) shows each step. Then use the checklist on page 4.

5 Manage dependencies carefully

- Commit your **lockfile** so everyone gets the same versions (table on page 4).
- Add only the libraries you actually use, and remove the ones you stopped using.
- Prefer well-known, maintained packages. Run your package manager's audit (`npm audit`, `pip-audit`, `cargo audit`, …) and update anything with known vulnerabilities.
- Respect licences. Keep licence notices for code you include, and add a `LICENSE` file.

✗ Don't commit `node_modules`, virtual environments, build output or copies of libraries.

6 Test the parts that matter

- Write a few **meaningful** tests for your core logic, such as calculations, rules, parsers, masking and risk thresholds. Quality beats quantity.
- Each test should assert on real outputs. A test that only checks the code doesn't crash proves very little.
- Cover edge cases: empty input, invalid values, boundary values and missing fields.
- Mock LLM and external API calls in unit tests so they run fast and give the same result every time.
- Put tests in the standard place for your stack (`tests/`, `__tests__/`, `src/test/`) and give the one command that runs them in your README.

7 Keep your repository clean

- Use Git from the start, and commit often with meaningful messages (for example "add premium calculator", not "update").
- Use a proper `.gitignore` for your stack.
- Use only synthetic or sample data. No real Aadhaar, PAN, card or patient records.

✗ Remove junk such as `.DS_Store`, `__pycache__`, logs, temporary files, old zips and "final_v2_copy" folders.

8 Handle errors gracefully

- Catch **specific** exceptions and handle them properly.
- Set a **timeout** on every network, database and API call. Retry with back-off when a failure is temporary.
- Show users a helpful message, and log the details for developers.
- Use a logging library rather than `print` / `console.log` for important events.
- Plan what happens when an external service, including an LLM, is slow or down: a fallback, a cached answer or a clear error.

✗ Never swallow errors silently with `except: pass` or an empty `catch {}`.

9 Using LLMs and AI APIs (if used)

Any provider, whether cloud or local, is allowed. Use it carefully.

- Keep model names, temperature and token limits in config, not scattered through your code.
- Store prompts in separate files or named constants so they are easy to find, review and change.
- Keep user input separate from system instructions. Treat uploaded files, retrieved documents and web pages as **untrusted**, because they can carry prompt-injection attacks.
- When your code consumes LLM output, ask for JSON and **validate** it, for example with Pydantic or Zod, before you use it.
- Set timeouts and token limits, handle rate limits and API errors, and rate-limit your own routes that call paid LLMs.
- Mask or remove personal data (names, phone numbers, Aadhaar, medical or financial details) before sending it to an external API, wherever you can.

✗ Never run LLM output directly as code, shell commands or SQL, or render it as raw HTML.

10 Agents, tools and MCP servers (if used)

Agents are powerful. Give them limits.

- Always cap an agent loop with a maximum number of steps, iterations or time.
- Give each tool only the access it needs: allow-list specific commands, folders, tables or URLs.
- Define tool inputs with a schema and validate them, and check tool results before acting on them.
- Ask a **human to confirm** risky actions such as payments, deleting or changing records, or sending messages.
- Protect MCP servers and webhooks with authentication and input checks. Pin the versions of third-party tools and servers you use.
- Log or trace agent steps and tool calls so you can see what happened.
- Use an agent where it genuinely helps. A simple, reliable pipeline is perfectly fine.

✗ No tool that gives an agent unrestricted shell, file or database access outside a sandbox or allow-list.

11 Data science, ML and RAG (if used)

- Fix random seeds. Split train and test data properly, with **no leakage**: never fit on test data, use time-based splits for time series, and keep each patient or customer in only one split.
- Compare against a simple baseline, and report metrics that suit the problem.
- Move reusable code out of notebooks into modules. Do a *Restart & Run All* before you submit.
- Load models safely: prefer `safetensors` or `weights_only=True`, and don't load pickle files or use `trust_remote_code=True` with untrusted sources.
- For RAG, keep chunking, embedding and retrieval settings in config, and return sources or citations with answers.

✗ Don't commit large datasets, model weights or real personal data. Use synthetic data plus a small sample or a download script.

12 Document your work: the README

- **What** it does, and which problem statement it solves.
- **How to run** it: exact setup and run commands, the environment variables needed (point to `.env.example`) and how to run the tests.
- **Architecture**: a short diagram or a list of the main components and how they connect.
- **AI usage**: models, APIs and agents in your app, plus the AI coding assistants you used.
- **Pre-built and third-party code**: what your team wrote for this hackathon before Day 1, and what came from open-source libraries, templates or starter kits.
- **Limitations** and known issues. Be honest.
- In the code, comments should explain *why*, not *what*. Add docstrings to key functions.

Fair play and originality

- Allowed: code written **for this hackathon** (even before Day 1), open-source libraries and frameworks, starter templates and AI coding assistants.
- **Declare** pre-built code, templates and copied snippets in your README, with licence notices. Your lockfile covers libraries.

✗ Not allowed: application code from your own or anyone else's past projects or other hackathons, even if it is public.

- Submissions are checked for similarity against each other and against common public sources. Undeclared copied code will be flagged.

✗ Don't copy another team's code or share yours with other teams.

- Every team member should be able to explain the code.

QUICK REFERENCE BY STACK

STACK	FORMATTER / LINTER	LOCKFILE TO COMMIT	TESTS	DON'T COMMIT
Python	`ruff` (lint + format) or `black` + `flake8`	`uv.lock` / `poetry.lock` / pinned `requirements.txt`	`pytest`	`.venv` / `venv`, `__pycache__`, `.ipynb_checkpoints`
JavaScript / TypeScript	ESLint + Prettier	`package-lock.json` / `pnpm-lock.yaml` / `yarn.lock` / `bun.lock`	Vitest / Jest	`node_modules`, `dist`, `.next`
Java	Checkstyle or google-java-format	Fixed versions in `pom.xml` / `gradle.lockfile` (if enabled)	JUnit 5/6	`target`, `build`
Go	`gofmt`, `go vet`	`go.mod` + `go.sum`	`go test`	compiled binaries
Rust	`rustfmt`, `clippy`	`Cargo.lock`	`cargo test`	`target`
.NET	`dotnet format` + built-in analysers	`packages.lock.json` (enable `RestorePackagesWithLockFile`)	xUnit / NUnit	`bin`, `obj`

Submission checklist: before you upload

☐ Commit everything, then zip a fresh `git clone` (keeps `.git`, leaves out `.env` and ignored files)

☐ Zip named `teamID_psID.zip`; all code in `FULL_CODE/`, your own `README.md` next to it

☐ No secrets: `.env` excluded, `.env.example` included

☐ No `node_modules`, virtual environments, build output, large data or model files

☐ Lockfile committed

☐ Formatter and linter run, and warnings fixed

☐ Tests included, with the command to run them

☐ README covers what, how to run, architecture, AI usage, pre-built code and limitations

☐ Notebooks run cleanly from top to bottom (if any)